

PROJECT REPORT



Cinema Audience Forecasting

Predicting Daily Theater Audience Counts with Machine Learning

MLP_PROJECT | ROHAN KUMAR

Repository: 23f3001514/MLP_PROJECT

Abstract

Cinema operators must plan staffing, concessions, and show scheduling around how many people will actually attend each screening. This project builds a supervised regression pipeline that forecasts `audience_count` for a given theater and show date using historical booking, visit, and theater-metadata records. Seven raw datasets are consolidated into a single modeling table, enriched with sixteen engineered features spanning calendar signals, per-theater statistics, lag values, rolling averages, and booking aggregates. Four regression algorithms — Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting — are trained and compared on a held-out validation split. Gradient Boosting emerges as the strongest model, achieving a validation RMSE of 18.03 and an R^2 of 0.656 after hyperparameter tuning via grid search. Final submission predictions are produced through a blending strategy that combines model output with theater-level historical averages and day-of-week seasonality, then clips results to a realistic percentile range.

1. Introduction

1.1 Problem Statement

Accurately predicting how many patrons will attend a given cinema show is valuable for staffing, inventory, and revenue planning. The task is framed as a tabular regression problem: given identifiers for a theater and a future show date, predict the expected `audience_count`. The dataset reflects realistic operational messiness — multiple overlapping data sources (a booking platform and a point-of-sale system), missing values, and no explicit future-looking features, all of which must be engineered from historical patterns.

1.2 Objectives

- Consolidate visit, booking, theater, and date data into a single, model-ready table.
- Engineer temporal and theater-level features that capture demand patterns.
- Train and benchmark multiple regression models on a common validation split.
- Select and tune the best-performing model.
- Generate realistic, bounded predictions for a submission file.

2. Dataset Description

The project draws on seven CSV files supplied for the Cinema Audience Forecasting challenge, summarized below.

Dataset	Description
booknow_visits	Audience count recorded per show, per theater (primary target source).
booknow_theaters	Theater metadata such as theater type and geographic area.
date_info	Calendar attributes associated with each show date.
booknow_booking	Individual booking and ticket transaction records, with timestamps.
cinePOS_booking	Point-of-sale booking records from a secondary system.
cinePOS_theaters	Theater metadata associated with the POS system.
sample_submission	Template defining the required prediction format (theater + date → <code>audience_count</code>).

Table 1. Source datasets used in the project.

Date fields (show_date, show_datetime, booking_datetime) are parsed into proper datetime types during loading, and dataset shapes and summary statistics (total records, unique theaters, mean audience) are printed to verify a successful load before any transformation begins.

3. Methodology

3.1 Data Loading and Preprocessing

All seven files are loaded with pandas, relevant date columns are converted to datetime, and basic diagnostics (row counts, unique theater counts, mean audience) are printed to sanity-check the raw inputs before merging.

3.2 Exploratory Data Analysis

Six visualizations are produced to build intuition about the data before modeling, covering:

- Distribution of audience count across all shows.
- Proportion of theaters by type (pie chart).
- Number of scheduled shows by day of week.
- Daily audience trend over time.
- Additional relationships between theater attributes and attendance.

These plots confirm that audience counts vary meaningfully by day of week and by theater, motivating the feature engineering strategy that follows.

3.3 Feature Engineering

Sixteen features are engineered from the merged dataset and grouped into six categories, summarized in Table 2. Lag and rolling features are computed per theater to respect the panel structure of the data, and missing values introduced by these operations (e.g., the first 14 days of a theater's history) are backfilled using the theater's own historical mean or standard deviation.

Category	Features
Calendar	day_of_week_num, month, day_of_month, quarter, week_of_year, is_weekend
Theater statistics	theater_mean, theater_median, theater_std, theater_min, theater_max, theater_count
Lag features	lag_1, lag_3, lag_7, lag_14 (audience count N days prior, per theater)
Rolling statistics	rolling_mean_3 / 7 / 14, rolling_std_7 / 14
Booking aggregates	total_tickets, avg_tickets, booking_count (per theater, per day)
Encoded categoricals	theater_type_encoded, theater_area_encoded (label-encoded)

Table 2. Engineered feature categories used for modeling.

Of the sixteen engineered signals, a curated subset of fifteen features is passed into the final feature_list used for model training, prioritizing the features with the strongest observed relationship to audience_count.

3.4 Train–Validation Split

Missing and infinite values in the feature matrix are replaced with zero, and the data is split 80/20 into training and validation sets, giving every model a common, fixed benchmark for comparison.

3.5 Model Training

Four regression models are trained on the identical training split and evaluated on the same validation set:

- Linear Regression — baseline linear model with no regularization.
- Ridge Regression — L2-regularized linear model ($\alpha = 1.0$).
- Random Forest Regressor — 150 trees, max depth 20.
- Gradient Boosting Regressor — 150 estimators, max depth 6.

Each model is scored using RMSE, MAE, and R^2 on the validation set, and the model with the lowest RMSE is selected as the best performer.

3.6 Hyperparameter Tuning

Since Gradient Boosting produced the strongest baseline result, it is further tuned with GridSearchCV using 3-fold cross-validation over a grid spanning `n_estimators`, learning rate, and tree depth, run on a subset of the training data to keep search time manageable. The tuned model's validation RMSE improves further over the untuned baseline.

4. Results and Model Comparison

Table 3 summarizes the four candidate models. Gradient Boosting achieved the lowest validation RMSE and highest R^2 both before and after tuning, and was selected as the final model.

Model	Configuration	Relative Performance
Linear Regression	Ordinary least squares	Baseline
Ridge Regression	$\alpha = 1.0$	Comparable to baseline
Random Forest	150 trees, max depth 20	Improved over linear models
Gradient Boosting	150 estimators, max depth 6 (tuned)	Best performer — selected model

Table 3. Model configurations compared during training.

Gradient Boosting Best Model	18.03 Validation RMSE	0.656 Validation R^2
--	---------------------------------	----------------------------------

Figure summary — final validation performance of the selected model.

Model comparison is further supported by four diagnostic plots: RMSE across models, R^2 across models, an actual-vs-predicted scatter plot for the best model, and a feature importance chart. The feature importance analysis confirms that theater-level statistics and short-horizon lag/rolling features (`theater_mean`, `lag_7`, `rolling_mean_7`) contribute the most predictive signal, while calendar features play a secondary, seasonal-adjustment role.

5. Prediction Strategy

Raw model output is not used directly for the final submission. Instead, predictions are refined through a three-step blending process designed to reduce variance and keep forecasts within realistic bounds:

- Blend with theater history: each raw prediction is combined as 70% model output + 30% the theater's historical mean audience, damping outlier predictions for theaters with limited recent data.

- Day-of-week adjustment: predictions are scaled by a factor derived from each day's historical audience ratio relative to the overall average, applied at 15% strength.
- Realistic bounds: final predictions are clipped to the 5th–95th percentile of historical audience_count, then rounded to the nearest integer.

This adjustment layer reflects a common practical pattern in forecasting pipelines: pure model output is regularized against strong, simple priors (historical averages and seasonality) to avoid unrealistic extrapolation on sparse or unseen theater/date combinations.

6. Conclusion

This project demonstrates a complete, practical forecasting pipeline: seven raw sources were consolidated, sixteen domain-informed features were engineered, four models were benchmarked under identical conditions, and the best model (Gradient Boosting) was tuned and wrapped in a blending strategy for robust final predictions. The result — a validation R^2 of 0.656 with RMSE of 18.03 — indicates the model explains a substantial share of audience variability while leaving room for further refinement, most notably around richer external signals such as holidays and local events.

7. Future Work

- Incorporate holiday, festival, and local-event calendars as explicit features.
- Evaluate gradient boosting libraries such as XGBoost, LightGBM, or CatBoost for potential accuracy gains.
- Adopt walk-forward (time-based) cross-validation in place of a single random split, to better reflect the forecasting use case.
- Explore stacking or weighted ensembling across the four trained models.
- Incorporate movie-level attributes (genre, release week, ratings) if such data becomes available.

Appendix — Technical Stack

The project is implemented in Python within a Jupyter notebook, using the following core libraries:

- pandas & numpy — data loading, merging, and feature computation
- matplotlib & seaborn — exploratory and comparison visualizations
- scikit-learn — model implementations, train/validation split, GridSearchCV, and evaluation metrics

Key output artifacts generated by the pipeline:

- eda_analysis.png — exploratory data analysis visualizations
- model_comparison.png — model performance comparison plots
- submission.csv — final audience_count predictions in the required submission format